

实验项目	实验二 基于卷积神经网络的手写数字识别实验		
实验时间	2026.4.28	实验地点	北理工 B507
一、实验目的 通过本实验使学生了解 MINIST 手写体数据集；掌握卷积神经网络的结构；熟练卷积神经网络的构建方法和训练过程。 二、实验环境 Python 编程环境。 三、实验内容 使用 Pytorch 框架构建卷积神经网络，加载 MINIST 手写体数据集，并进行预处理，设置合适的训练次数、批次和学习率等相关参数，完成训练和评估，观察实验结果并进行分析。 三、实验步骤与结果 (1)导入相关库：torch, Variable,DataLoader,torch.nn,functional,optim 等 ①实验代码 <pre>import torch from torch.autograd import Variable import torchvision from torch.utils.data import DataLoader</pre> (2) 数据加载和预处理：定义超参数，加载 MNIST 数据集得到 train_loader 和 test_loader，并展示数字图像。 ①实验代码和结果 <pre>train_loader = torch.utils.data.DataLoader(torchvision.datasets.MNIST(r"D:\MNIST\MNIST", train=True, download=True, transform=torchvision.transforms.Compose([torchvision.transforms.ToTensor(), torchvision.transforms.Normalize((0.1307,),(0.3081,))])), batch_size=batch_size_train, shuffle=True) test_loader = torch.utils.data.DataLoader(</pre>			

```

torchvision.datasets.MNIST(r"D:\MNIST\MNIST",
                           train=False,
                           download=True,
                           transform=torchvision.transforms.Compose([
                               torchvision.transforms.ToTensor(),
                               torchvision.transforms.Normalize((0.1307,), (0.3081,))]), batch_size=batch_size_test, shuffle=
True)

```



②简述返回内容

代码返回 MNIST 数据集的训练数据加载器 `train_loader` 和测试数据加载器 `test_loader`，完成数据加载、张量转换、标准化预处理，并实现数据分批与打乱，为后续模型训练和测试提供数据支撑。

(3)构建网络：使用两个 Conv2d 卷积层和两个全连接层 Linear，并且选择整流线性单元（简称 ReLU）作为激活函数以及使用两个 Dropout 层

①实验代码

```

import torch.nn as nn

import torch.nn.functional as F

import torch.optim as optim

class Net(nn.Module):

    def __init__(self):

        super(Net, self).__init__()

        self.conv1 = nn.Conv2d(1, 10, kernel_size=5)

```

```

self.conv2 = nn.Conv2d(10, 20, kernel_size=5)

self.conv2_drop = nn.Dropout2d()

self.fc1 = nn.Linear(320, 50)

self.fc2 = nn.Linear(50, 10)

def forward(self, x):

    x = F.relu(F.max_pool2d(self.conv1(x), 2))

    x = F.relu(F.max_pool2d(self.conv2_drop(self.conv2(x)), 2))

    x = x.view(-1, 320)

    x = F.relu(self.fc1(x))

    x = F.dropout(x, training=self.training)

    x = self.fc2(x)

    return F.log_softmax(x)

```

②输出构建网络参数截图

F:\Anaconda_envs\envs\pytorch2.3.1\python.exe "C:\Users\28614\Downloads\实验二 手写汉字识别\手写汉字识别.py"

```

-----
Layer (type)          Output Shape          Param #
-----
Conv2d-1              [-1, 6, 62, 62]       60
MaxPool2d-2           [-1, 6, 31, 31]       0
Conv2d-3              [-1, 16, 27, 27]      2,416
MaxPool2d-4           [-1, 16, 13, 13]      0
Linear-5              [-1, 512]             1,384,960
Linear-6              [-1, 84]              43,092
Linear-7              [-1, 100]             8,500
=====
Total params: 1,439,028
Trainable params: 1,439,028
Non-trainable params: 0
-----

Input size (MB): 0.02
Forward/backward pass size (MB): 0.33
Params size (MB): 5.49
Estimated Total Size (MB): 5.84

```

(4)训练网络：定义训练函数和测试函数

①实验代码

```

train_losses = []

train_counter = []

test_losses = []

```

```

test_counter = [i * len(test_loader.dataset) for i in range(0, n_epochs + 1)]

def train(epoch):
    network.train()

    for batch_idx, (data, target) in enumerate(train_loader):
        optimizer.zero_grad()
        output = network(data)
        loss = F.nll_loss(output, target)
        loss.backward()
        optimizer.step()

        if batch_idx % log_interval == 0:
            print('Train Epoch: {} [{}/{}] ({:.0f}%) \t Loss: {:.6f}'.format(
                epoch, batch_idx * len(data), len(train_loader.dataset),
                100. * batch_idx / len(train_loader), loss.item()))
            train_losses.append(loss.item())
            train_counter.append(
                (batch_idx * 64) + ((epoch - 1) * len(train_loader.dataset)))
            torch.save(network.state_dict(), './tmp/model.pth')
            torch.save(optimizer.state_dict(), './tmp/optimizer.pth')

train(1)

def test():
    network.eval()

    test_loss = 0
    correct = 0

    with torch.no_grad():
        for data, target in test_loader:
            output = network(data)
            test_loss += F.nll_loss(output, target, size_average=False).item()
            pred = output.data.max(1, keepdim=True)[1]
            correct += pred.eq(target.data.view_as(pred)).sum()

    test_loss /= len(test_loader.dataset)

```

```

test_losses.append(test_loss)

print('\nTest set: Avg. loss: {:.4f}, Accuracy: {}/{} ({:.0f}%)'\n'.format(
    test_loss, correct, len(test_loader.dataset), 100.*correct/ len(test_loader.dataset)))

```

②训练结果

```

Train Epoch: 5 [48640/60000 (81%)]    Loss: 0.075619
Train Epoch: 5 [49280/60000 (82%)]    Loss: 0.064978
Train Epoch: 5 [49920/60000 (83%)]    Loss: 0.090977
Train Epoch: 5 [50560/60000 (84%)]    Loss: 0.063232
Train Epoch: 5 [51200/60000 (85%)]    Loss: 0.272390
Train Epoch: 5 [51840/60000 (86%)]    Loss: 0.212916
Train Epoch: 5 [52480/60000 (87%)]    Loss: 0.135088
Train Epoch: 5 [53120/60000 (88%)]    Loss: 0.116652
Train Epoch: 5 [53760/60000 (90%)]    Loss: 0.344871
Train Epoch: 5 [54400/60000 (91%)]    Loss: 0.369638
Train Epoch: 5 [55040/60000 (92%)]    Loss: 0.263402
Train Epoch: 5 [55680/60000 (93%)]    Loss: 0.192600
Train Epoch: 5 [56320/60000 (94%)]    Loss: 0.198391
Train Epoch: 5 [56960/60000 (95%)]    Loss: 0.075034
Train Epoch: 5 [57600/60000 (96%)]    Loss: 0.128487
Train Epoch: 5 [58240/60000 (97%)]    Loss: 0.242869
Train Epoch: 5 [58880/60000 (98%)]    Loss: 0.307537
Train Epoch: 5 [59520/60000 (99%)]    Loss: 0.377949

```

```

Test set: Avg. loss: 0.0684, Accuracy: 9783/10000 (98%)

```

(5)评估网络：画出训练中损失的变化曲线，并随机显示预测结果。

①实验代码

```

test()

for epoch in range(1, n_epochs + 1):
    train(epoch)
    test()

import matplotlib.pyplot as plt
fig = plt.figure()
plt.plot(train_counter, train_losses, color='blue')
plt.plot(test_counter, test_losses, color='red', linestyle=':')
plt.legend(['训练损失', '测试损失'], loc='upper right')
plt.xlabel('训练次数')
plt.ylabel('损失')
plt.savefig('../tmp/1.png', dpi=1080)
plt.show()

```

```

# 预测结果

examples = enumerate(test_loader)

batch_idx, (example_data, example_targets) = next(examples)

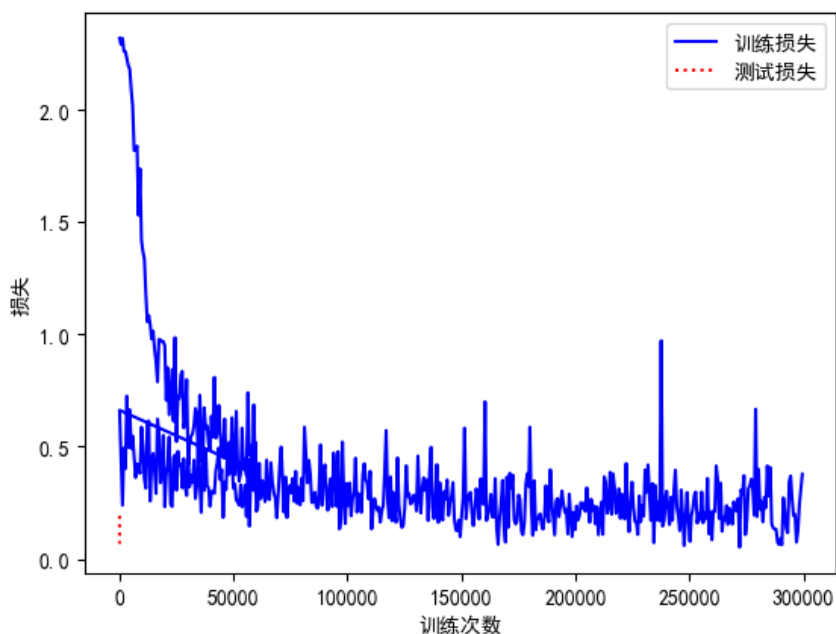
with torch.no_grad():
    output = network(example_data)

fig = plt.figure()

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.tight_layout()
    plt.imshow(example_data[i][0], cmap='gray', interpolation='none')
    plt.title('预测结果: {}'.format(
        output.data.max(1, keepdim=True)[1][i].item()))
    plt.xticks([])
    plt.yticks([])

plt.show()

```



②观察损失曲线，阐述其含义？

该曲线展示了模型训练过程中损失值的变化：训练损失前期快速下降，中后期波动下降并趋于稳定，说明模型有效学习并收敛；测试损失初期数值较低，未出现过拟合迹象，模型泛化能力良好。

四、思考题

(1) MNIST 数据集的特点？

(2) 构建网络时，将激活函数换成 Sigmoid 函数，代码应该如何修改？

(3) 简述 softmax 函数的功能？

实验成绩评定表

序号	考核项目	分值分布	成绩
1	预习情况	20	
2	实验任务完成情况	40	
3	实验报告质量	40	
总分			
指导教师签字			